In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
df = pd.read_csv("Advertising.csv")
dfs = df.iloc[:20,:]
dfs
```

Out[2]:

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9
5	6	8.7	48.9	75.0	7.2
6	7	57.5	32.8	23.5	11.8
7	8	120.2	19.6	11.6	13.2
8	9	8.6	2.1	1.0	4.8
9	10	199.8	2.6	21.2	10.6
10	11	66.1	5.8	24.2	8.6
11	12	214.7	24.0	4.0	17.4
12	13	23.8	35.1	65.9	9.2
13	14	97.5	7.6	7.2	9.7
14	15	204.1	32.9	46.0	19.0
15	16	195.4	47.7	52.9	22.4
16	17	67.8	36.6	114.0	12.5
17	18	281.4	39.6	55.8	24.4
18	19	69.2	20.5	18.3	11.3
19	20	147.3	23.9	19.1	14.6

In [3]:
```
# Store data into numpy array
mat = dfs.iloc[:,1:5].to_numpy()
mat
```

Out[3]:
```
array([[230.1,  37.8,  69.2,  22.1],
       [ 44.5,  39.3,  45.1,  10.4],
       [ 17.2,  45.9,  69.3,   9.3],
       [151.5,  41.3,  58.5,  18.5],
       [180.8,  10.8,  58.4,  12.9],
       [  8.7,  48.9,  75. ,   7.2],
       [ 57.5,  32.8,  23.5,  11.8],
       [120.2,  19.6,  11.6,  13.2],
       [  8.6,   2.1,   1. ,   4.8],
       [199.8,   2.6,  21.2,  10.6],
       [ 66.1,   5.8,  24.2,   8.6],
       [214.7,  24. ,   4. ,  17.4],
       [ 23.8,  35.1,  65.9,   9.2],
       [ 97.5,   7.6,   7.2,   9.7],
       [204.1,  32.9,  46. ,  19. ],
       [195.4,  47.7,  52.9,  22.4],
       [ 67.8,  36.6, 114. ,  12.5],
       [281.4,  39.6,  55.8,  24.4],
       [ 69.2,  20.5,  18.3,  11.3],
       [147.3,  23.9,  19.1,  14.6]])
```

In [4]:
```
# Standardize data by z-score
column_means = np.mean(mat, axis = 0)
column_std = np.std(mat, axis = 0) # pandas and numpy use slightly different standa

mat_standard = (mat - column_means)/column_std # z-score to standardize data
mat_standard
```

Out[4]:
```
array([[ 1.35032393,  0.66836934,  0.94254628,  1.62501806],
       [-0.91179468,  0.7680268 ,  0.10711541, -0.58447773],
       [-1.24453088,  1.20651961,  0.9460128 , -0.79220811],
       [ 0.3923362 ,  0.90090341,  0.57162884,  0.9451732 ],
       [ 0.74944867, -1.12546488,  0.56816232, -0.11236325],
       [-1.34813006,  1.40583453,  1.14360433, -1.18878427],
       [-0.75334888,  0.33617782, -0.64165251, -0.32009362],
       [ 0.01084744, -0.5408078 , -1.05416816, -0.05570951],
       [-1.34934888, -1.70347813, -1.42161909, -1.64201418],
       [ 0.98102331, -1.67025898, -0.72138242, -0.54670857],
       [-0.64853088, -1.4576564 , -0.61738688, -0.92440016],
       [ 1.16262659, -0.24847926, -1.31762354,  0.73744283],
       [-1.16408916,  0.48898592,  0.82815118, -0.81109269],
       [-0.26582331, -1.33806746, -1.20669496, -0.71666979],
       [ 1.03343231,  0.34282165,  0.13831407,  1.0395961 ],
       [ 0.9273955 ,  1.32610856,  0.37750382,  1.6816718 ],
       [-0.62781104,  0.58864338,  2.49554641, -0.18790157],
       [ 1.97557546,  0.78795829,  0.47803285,  2.05936338],
       [-0.61074765, -0.48101333, -0.82191145, -0.41451652],
       [ 0.34114601, -0.25512309, -0.7941793 ,  0.2086746 ]])
```

In [5]:
```
# Objective function is the mean squared error
# f(m, b) = (1/n) * (sum (( m * x[i] + b) - y[i]) ** 2 )
# Substitute standardized data into mean squared error
# w is array of 4 values each representing m1, m2, m3, b
def f(w):

    sales_pred = np.dot(mat_standard[:,0:3], w[0:3]) + w[3]
```

```
    return ((sales_pred - mat_standard[:,3]) ** 2).mean() # mean squared of (y_pred

print(f([0, 0, 0, 0]))
```

```
1.0000000000000002
```

In [6]:
```
# Gradient: Partial derivative
# w is an array of 4 inputs, each represent initial guess for m1, m2, m3, b
def f1(w):

    # w = np.array([m1, m2, m3])
    sales_pred = np.dot(mat_standard[:,0:3], w[0:3]) + w[3]
    error = sales_pred - mat_standard[:,3] # [(m * x_i + b) - y_i]

    # insert column of 1's to calculate minimum b (design this way because of matri
    temp = np.insert(mat_standard[:,0:3], 3, 1, axis=1)

    # Partial derivatives
    # np.dot(a,b) is sum from n=1 to n=i of (a_i * b_i)
    # temp.shape[0] is the number of rows = number of datapoints
    gradient = 2 * np.dot(error, temp) / temp.shape[0]

    return gradient

f1([0,0,0,0])
```

Out[6]:
```
array([-1.72584963e+00, -8.59220897e-01, -4.09063908e-01,  4.44089210e-16])
```

In [7]:
```
# Gradient descent
gradient_variables = [0,0,0,0]
alpha = 0.1

for i in range(5000):

    gradient_variables = gradient_variables - alpha * f1(gradient_variables)

    if i % 500 == 0:
        print(f"Iteration {i} - Mean squared error: ", f(gradient_variables))
```

```
Iteration 0 - Mean squared error:  0.6547702383757598
Iteration 500 - Mean squared error:  0.0721357766608943
Iteration 1000 - Mean squared error:  0.07213577666089432
Iteration 1500 - Mean squared error:  0.07213577666089432
Iteration 2000 - Mean squared error:  0.07213577666089432
Iteration 2500 - Mean squared error:  0.07213577666089432
Iteration 3000 - Mean squared error:  0.07213577666089432
Iteration 3500 - Mean squared error:  0.07213577666089432
Iteration 4000 - Mean squared error:  0.07213577666089432
Iteration 4500 - Mean squared error:  0.07213577666089432
```

In [8]:
```
# Reverse engineer to get true values of m1, m2, m3, b
min_values = (gradient_variables[0:3] * column_std[3]) / column_std[0:3]
temp = gradient_variables[0:3] * mat[:,3].std() * np.mean(mat[:, 0:3], axis=0)
temp = temp / column_std[0:3]
b = mat[:,3].mean() + mat[:,3].std() * gradient_variables[3] - temp[0] - temp[1] -
```

```
print("m1: ", min_values[0],
      "\nm2: ", min_values[1],
      "\nm3: ", min_values[2],
      "\nb: ", b)
```

```
m1:  0.05524409995138
m2:  0.1688556765459869
m3:  -0.015225252984653001
b:  2.8593828453004475
```

In [9]:
```
# Check with minimize function
from scipy.optimize import minimize

# Brought over from previous code
tv = dfs.iloc[:,1]
radio = dfs.iloc[:,2]
news = dfs.iloc[:,3]
sales = dfs.iloc[:,4]

# Objective function: Mean squared error
def fMin(w):
    m1, m2, m3, b = w
    sales_pred = (m1 * tv) + (m2 * radio) + (m3 * news) + b
    return ((sales_pred - sales) ** 2).mean()

res = minimize(fMin, x0 = [0, 0, 0, 0])
res.x
```

Out[9]:
```
array([ 0.05524408,  0.16885569, -0.01522528,  2.85938665])
```